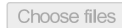


```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
from google.colab import files
files.upload()
```

 dataset.zip
dataset.zip(application/x-zip-compressed) - 397172539 bytes, last modified: 14/01/2026 - 100% done
 Saving dataset.zip to dataset.zip

```
import zipfile

with zipfile.ZipFile("dataset.zip", "r") as zip_ref:
    zip_ref.extractall("/content/")

print("Dataset extracted")
```

Dataset extracted

```
import os

os.listdir("/content/data")
```

['val', 'train']

```
os.listdir("/content/data/val")
```

['NonDemented', 'MildDemented', 'ModerateDemented', 'VeryMildDemented']

```
os.listdir("/content/data/train")
```

['NonDemented', 'MildDemented', 'ModerateDemented', 'VeryMildDemented']

```
!pip install -q tensorflow scikit-learn matplotlib seaborn
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Model
from tensorflow.keras.layers import GlobalAveragePooling2D

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import LinearSVC
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```
img_size = 224
batch_size = 32

train_dir = "/content/data/train"
val_dir = "/content/data/val"
```

```
datagen = ImageDataGenerator(rescale=1./255)
```

```

train_gen = datagen.flow_from_directory(
    train_dir,
    target_size=(224,224),
    batch_size=batch_size,
    class_mode="categorical",
    shuffle=False
)

val_gen = datagen.flow_from_directory(
    val_dir,
    target_size=(224,224),
    batch_size=batch_size,
    class_mode="categorical",
    shuffle=False
)

```

Found 33984 images belonging to 4 classes.
Found 6400 images belonging to 4 classes.

```
print(train_gen.class_indices)
```

```
{'MildDemented': 0, 'ModerateDemented': 1, 'NonDemented': 2, 'VeryMildDemented': 3}
```

```

base_model = MobileNetV2(
    weights="imagenet",
    include_top=False,
    input_shape=(224,224,3)
)

x = GlobalAveragePooling2D()(base_model.output)
feature_model = Model(inputs=base_model.input, outputs=x)

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_order0_weights_20160814_12.5/9406464/9406464 0s 0us/step

```

X_train = feature_model.predict(train_gen, verbose=1)
X_val = feature_model.predict(val_gen, verbose=1)

y_train = train_gen.classes
y_val = val_gen.classes

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
self._warn_if_super_not_called()
1062/1062 1379s 1s/step
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
self._warn_if_super_not_called()
200/200 258s 1s/step

```

```

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)

```

```

pca = PCA(n_components=128)
X_train = pca.fit_transform(X_train)
X_val = pca.transform(X_val)

```

```

classifiers = {
    "Linear SVM": LinearSVC(max_iter=3000),
    "Logistic Regression": LogisticRegression(max_iter=2000),
    "Random Forest": RandomForestClassifier(n_estimators=60, max_depth=12)
}

```

```
results = []
```

```
transfer_model_name = "MobileNetV2"
```

```
for name, model in classifiers.items():
    print("Training:", name)

    model.fit(X_train, y_train)
    preds = model.predict(X_val)

    acc = accuracy_score(y_val, preds) * 100

    results.append([transfer_model_name, name, acc])

    print(f"{transfer_model_name} + {name} Accuracy: {acc:.2f}%")
```

```
Training: Linear SVM
MobileNetV2 + Linear SVM Accuracy: 60.48%
Training: Logistic Regression
MobileNetV2 + Logistic Regression Accuracy: 61.45%
Training: Random Forest
MobileNetV2 + Random Forest Accuracy: 63.33%
```

```
df = pd.DataFrame(
    results,
    columns=["Transfer Model", "Classifier", "Accuracy (%)"]
)

df
```

	Transfer Model	Classifier	Accuracy (%)	
0	MobileNetV2	Linear SVM	60.484375	
1	MobileNetV2	Logistic Regression	61.453125	
2	MobileNetV2	Random Forest	63.328125	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
plt.figure(figsize=(8,5))
sns.barplot(data=df, x="Classifier", y="Accuracy (%)", hue="Transfer Model")
plt.title("Alzheimer Prediction using Transfer Learning + ML Classifiers")
plt.ylim(50, 80)
plt.show()
```

